ELTE
FACULTY OF
INFORMATICS

PROGRAMMING
Lecture 9

Zsuzsa Pluhár
pluharzs@inf.elte.hu

More programming patterns

Using more patterns

Complex problems – we need more patterns

- pattern as a function -> combine them

maximum selection + MIS

List all maximum elements of an integer sequence.

```
In: n∈N, x∈Z[1..n]
Aux: maxval∈Z
Out: cnt∈N, maxi∈N[1..n]
Pre: n>0
Post: (,maxval)=MAX(i=1..n,x[i]) and
    (cnt,maxi)=MIS(i=1..n,x[i]=maxval,i)
```

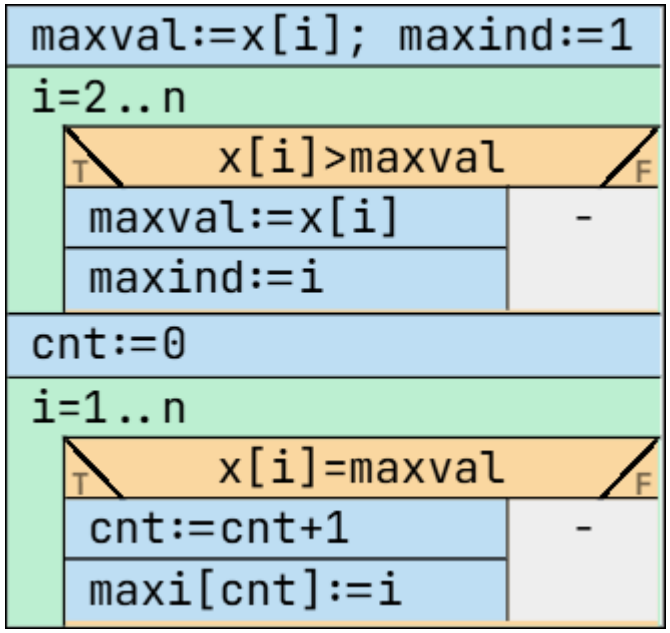
1st step: find the maximum value

maxval = helper variable

2nd step: list all items that equal the max.value

maximum selection + MIS

List all maximum elements of an integer sequence.



the result is correct,
but not effective

Program transformation

Goal, structure

An operation, that takes an algorithm (or computer program) and generates another algorithm (program) **semantically equivalent** to the original.

- Increase effectiveness;
- Simplify;
- Make it more feasible.

Program transformation – simplying, increase effectiveness

Problem: the fartherst point from the origin…

max:=1; maxval:=sqrt(p[1].x^2+p[1].y^2)	
i=2..N	
sqrt(p[i].x^2+p[i].y^2)>maxval	
max:=i	—
maxval:=sqrt(p[i].x^2+p[i].y^2)	

The square root is a monotonuos function – we don't use the maxval (not output data) -> doesn't disturbe the specification.

Program transformation – simplying, increase effectiveness,

Problem: the fartherst point from the origin…

max:=1; maxval:=p[1].x^2+p[1].y^2		
	i=2..N	
	distance:=p[i].x^2+p[i].y^2	
	distance>maxval	
	max:=i	—
	maxval:=distance	

we calculate the **same** expression several times -> helper variable.

Program transformation – expanding paralell value assignment

```
a,b,c:=f(x),g(x),h(x);
```

Can be divided into consecutive calculation, if the relation is ***cycle free***

```
a:=f(x); b:=g(x); c:=h(x);
```

Program transformation – expanding paralell value assignment

```
a,b,c:=b,c,a;
```

Can be divided into consecutive calculations with the help of an ***auxiliary variable***, if the relation is not cycle free – **contains a cycle**

```
av:=a; a:=b; b:=c; c:=av;
```

Program transformation – expanding parallel value assignment

```
a,b,c:=f(a,b,c);g(a,b,c);h(a,b,c)
```

Can be divided into consecutive calculations with the help of an ***auxiliary variable***, if the relation is not cycle free – **contains a cycle – more general**

```
ava:=a; avb:=b; avc:=c;
a:=f(ava,avb,avc); b:=g(ava,avb,avc); c:=h(ava,avb,avc);
```

Program transformation – function composition

```
b:=g(a);c:=f(b)
```

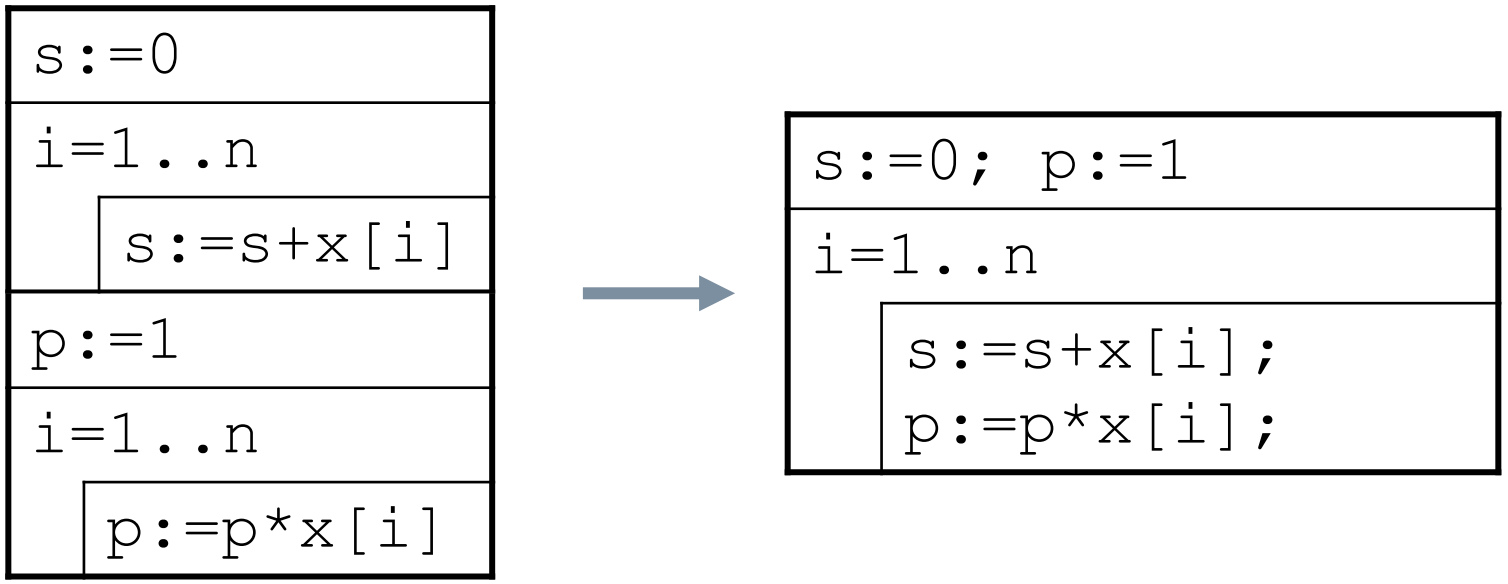
If

- the f function doesn't change the value of the parameter and
- w don't need the value of b later

```
c:=f(g(a))
```

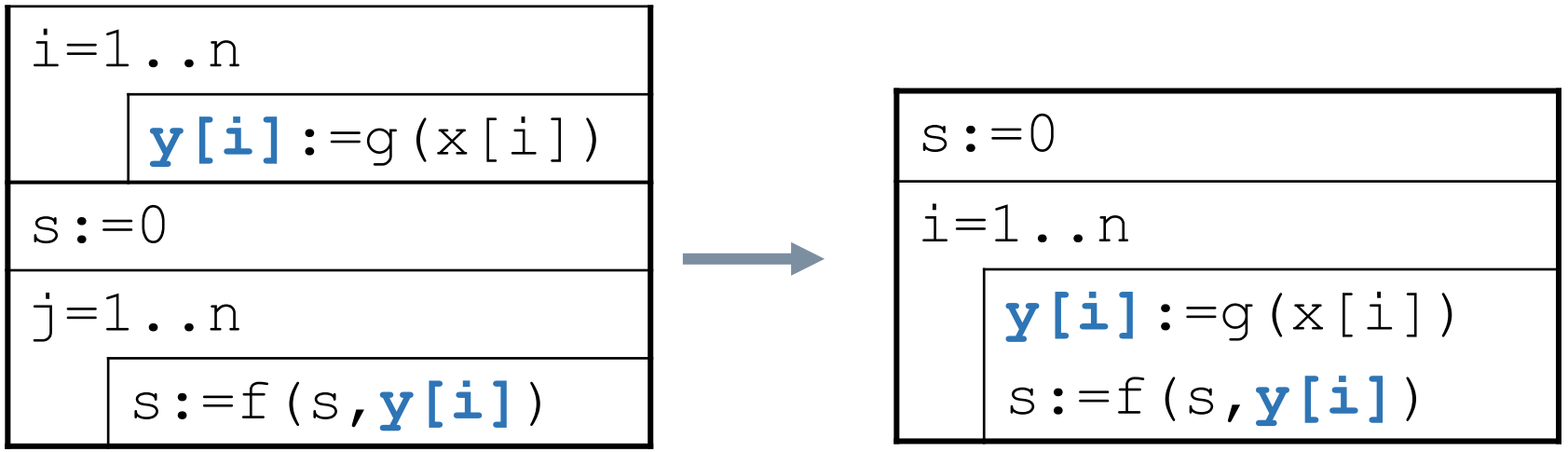
Program transformation – combining loops, loop fusion

Loops having the *same number of loop cycles (steps)*, can be combined, if they *are independent* of each other



Program transformation – combining loops, loop fusion

Loops having the *same number of loop cycles (steps)*, can be combined, if they *have week/poor connection* of each other



the *i. output* of the 1st loop can be the *i>j. input* of the 2nd loop, but the i. output of the 2nd loop cannot be the j>i. input of the 1st loop

Program transformation – combining loops, loop fusion

counterexample: expected value (m) and standard deviation (s)

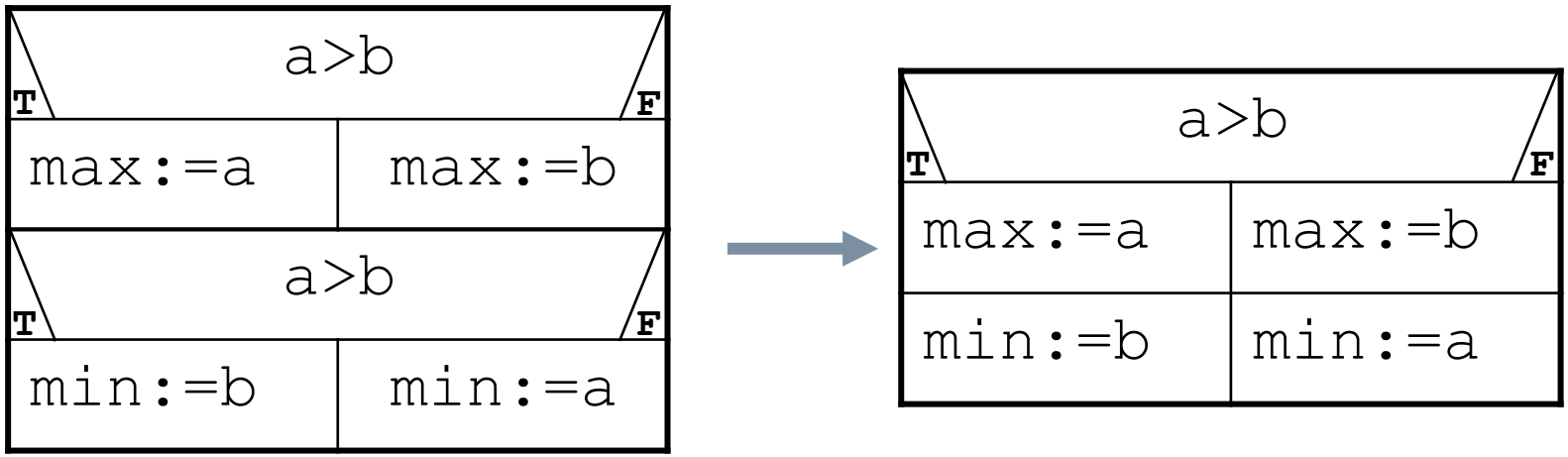
```
m:=0
        i=1..n
    m:=m+x[i]
m:=m/n
s:=0
i=1..n
  s:=s+(m-x[i])^2
s:=sqrt(s/n)
```

```
m:=0; s:=0
i=1..n
    m:=m+x[i]
    s:=s+(m-x[i])^2
m:=m/n; s:=sqrt(s/n)
```

the value of **m** is not ready to use!

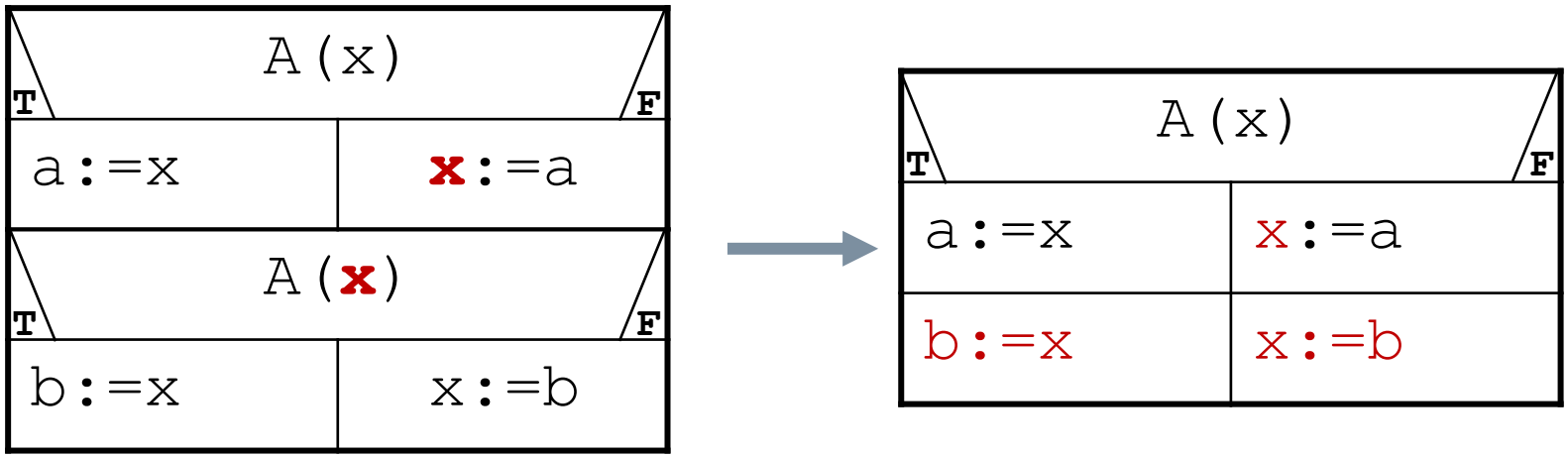
Program transformation – combining conditional statements

Conditional statements having the *same conditions* can be combined, if they are *independent* on each other



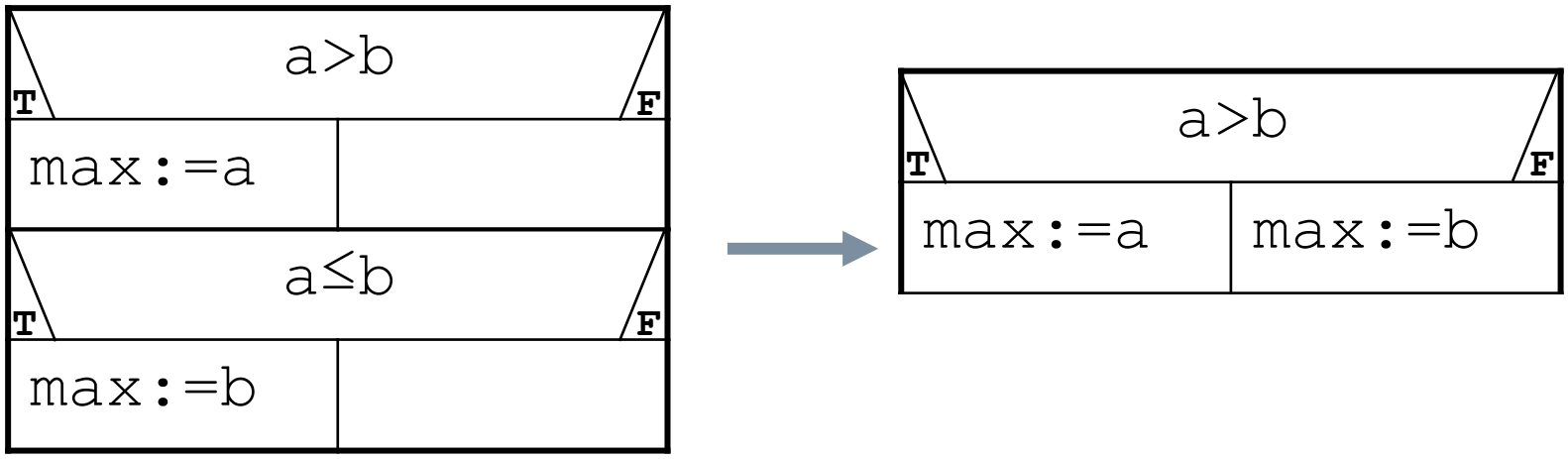
Program transformation – combining loops, loop fusion

counterexample: the value of the x will be changed until the 2nd conditional statement



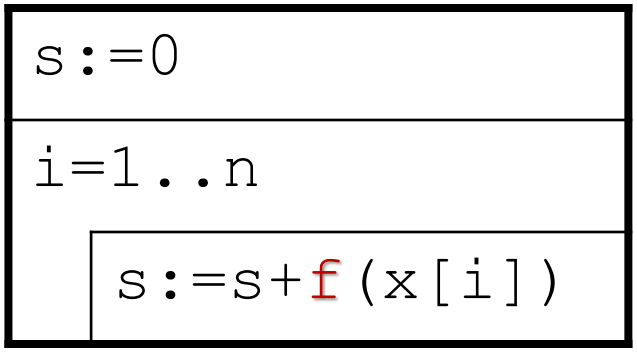
Program transformation – combining conditional statements

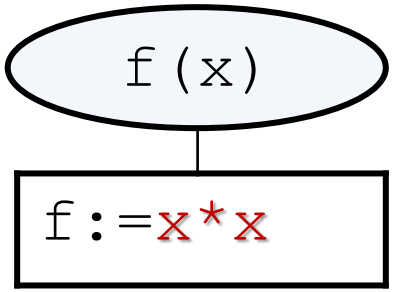
Combination of conditionals having *exclusive, full conditionals*, if they are *independent* of each other

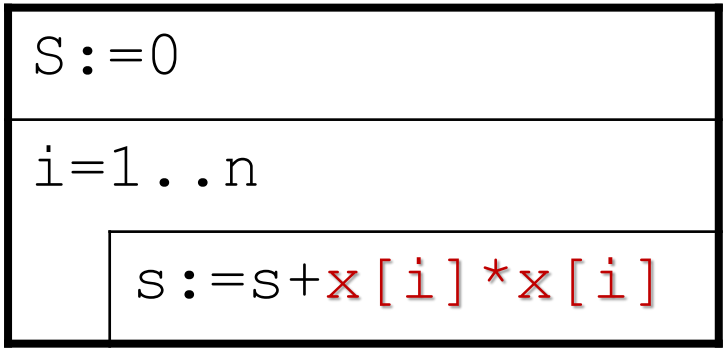


Program transformation – inlining function

Instead of a function call, the formula of a simple function (the body of the function) can be written. (C++ compilers can do such optimalization.)

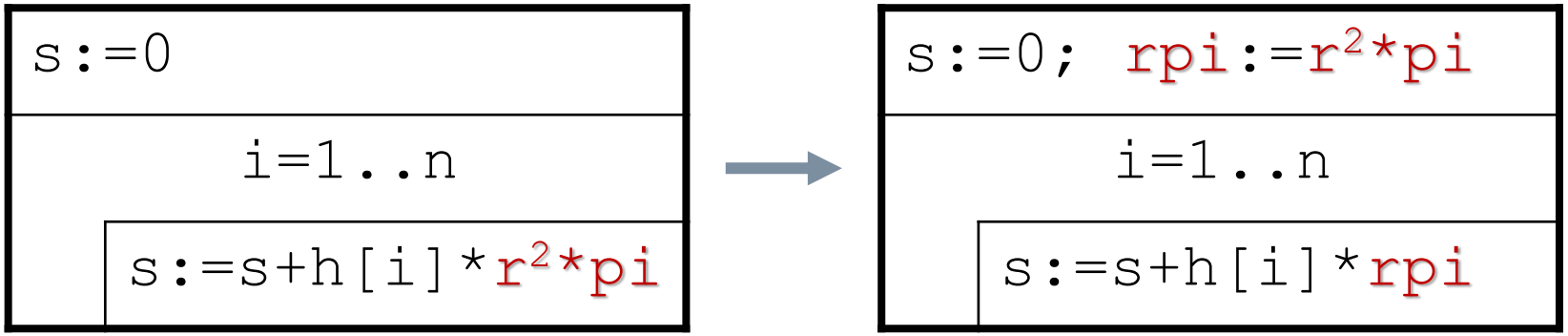






Program transformation – hoisting: moving expression outside loop

Loop-invariant expressions can be hoisted *out of loops*, thus improving run-time performance by *executing the expression only once* rather than at each iteration.



Program transformation

search, decide -> select

We put a special element with A attribute at the end of the sequence – certainly will be find one

```
i:=1
  i≤n and A(x[i])
    i:=i+1
exists:=i≤n
```

```
i:=1;
x[n+1]:=Titem
        A(x[i])
    i:=i+1
exists:=i≤n
```

Using the program transformations Patterns and constructions

Construction by copy

Construction by copy works for all patterns of algorithms.

Instead of the i-th elements of the `x` sequence in the input, you only have to write `f(x[i])`

… with **summation:**

```
SUM(i=1..n,x[i]) → SUM(i=1..n,f(x[i]))
```

… with **maximum selection**:

```
MAX(i=1..n,x[i]) → MAX(i=1..n,f(x[i]))
```

… with **MIS**:

```
MIS(i=1..n,A(x[i]),x[i]) → MIS(i=1..n,A(x[i]),f(x[i]))
```

The f transforming is applied to the input sequence of the outher pattern.

copy + summation

Problem: determine the sum of the with f transformed elements of the sequence x.

Solution: the „basis” will be the summation pattern

Let’s proof the correctness of

```
SUM(i=1..n,x[i]) → SUM(i=1..n,f(x[i]))
```

copy + summation

Problem: determine the sum of the with f transformed elements of the sequence x.

```
In: n∈N, x∈H1[1..n]
Fn: f:H1->H2
Out: s∈H2
Pre: -
Post: s=SUM(i=1..n,f(x[i]))
```

H1, H2: Set of numbers (N,Z,R,...)

copy + summation

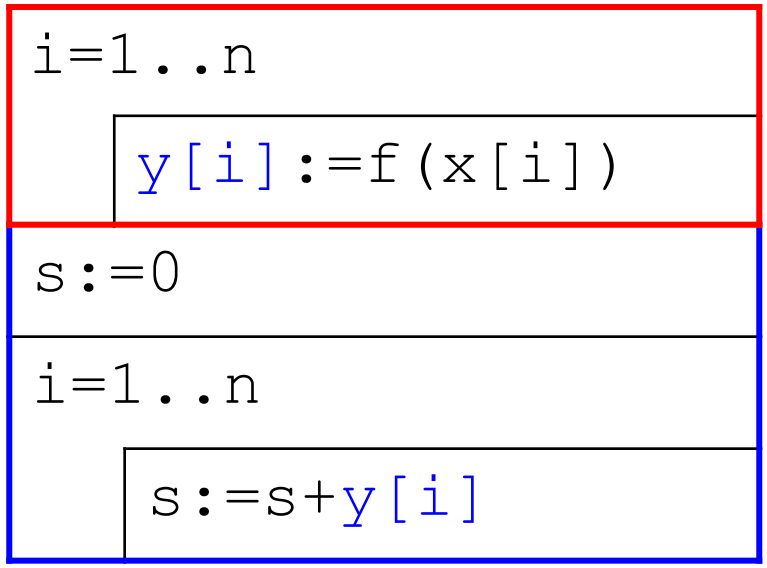
Problem: determine the sum of the with f transformed elements of the sequence x.

```
In: n∈N, x∈H1[1..n]
Aux: y∈H2[1..n]
Fn: f:H1->H2
Out: s∈H2
Pre: -
Post: y=COPY(i=1..n, f(x[i])) and
      s=SUM(i=1..n,y[i])
```

copy + summation

Problem: determine the sum of the with f transformed elements of the sequence x.

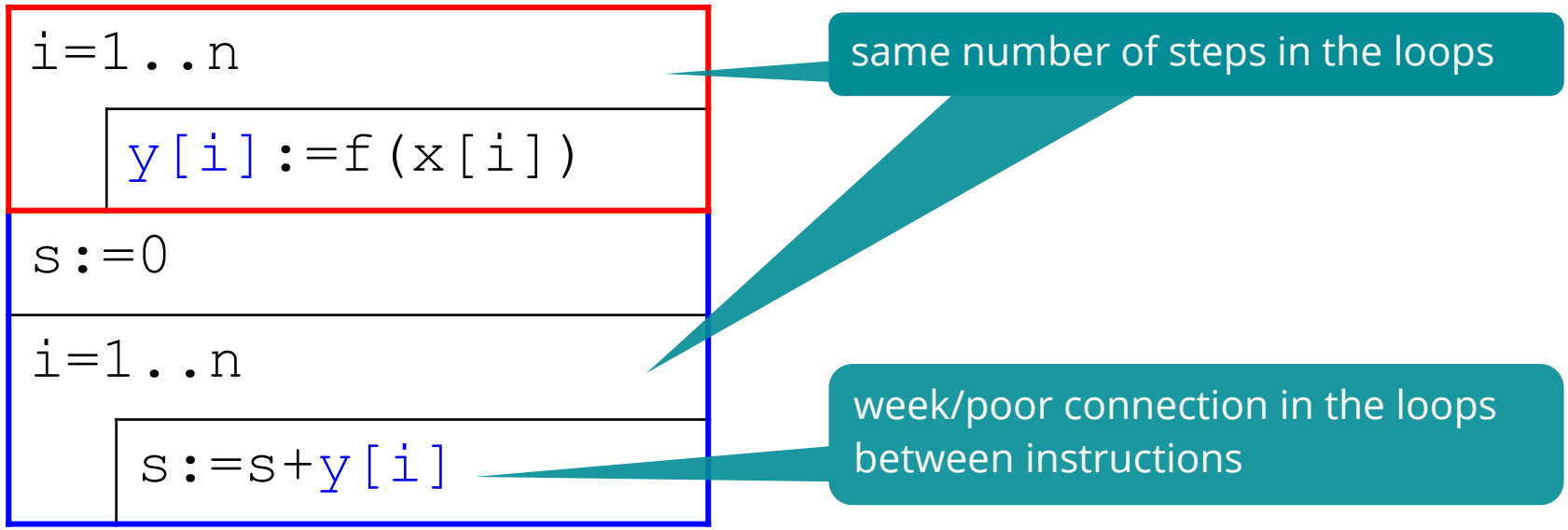
```
Post: y=COPY(i=1..n, f(x[i])) and
      s=SUM(i=1..n,y[i])
```



NB: the result is correct, but the algorithm is not really effective

copy + summation

Let's apply program transformations



copy + summation

Loop fusion with week connection…

```
s:=0
i=1..n
    y[i]:=f(x[i])
    s:=s+y[i]
```

copy + summation

Function composition

B:=*g*(A); C:=*f*(B)

↔

C:=*f*(*g*(A))

```
s:=0
i=1..n
    y[i]:=f(x[i])
    s:=s+y[i]
```

```
s:=0
i=1..n
    s:=s+f(x[i])
```

Q.E.D.

=> s=SUM(i=1..n,f(x[i]))

MIS + summation

Problem: Sum of elements with a certain attribute – conditional summation.

```
In: n∈N, x∈H[1..n]
Out: s∈H
Pre: -
Post: s=SUM(i=1..n,x[i],A(x[i]))
```

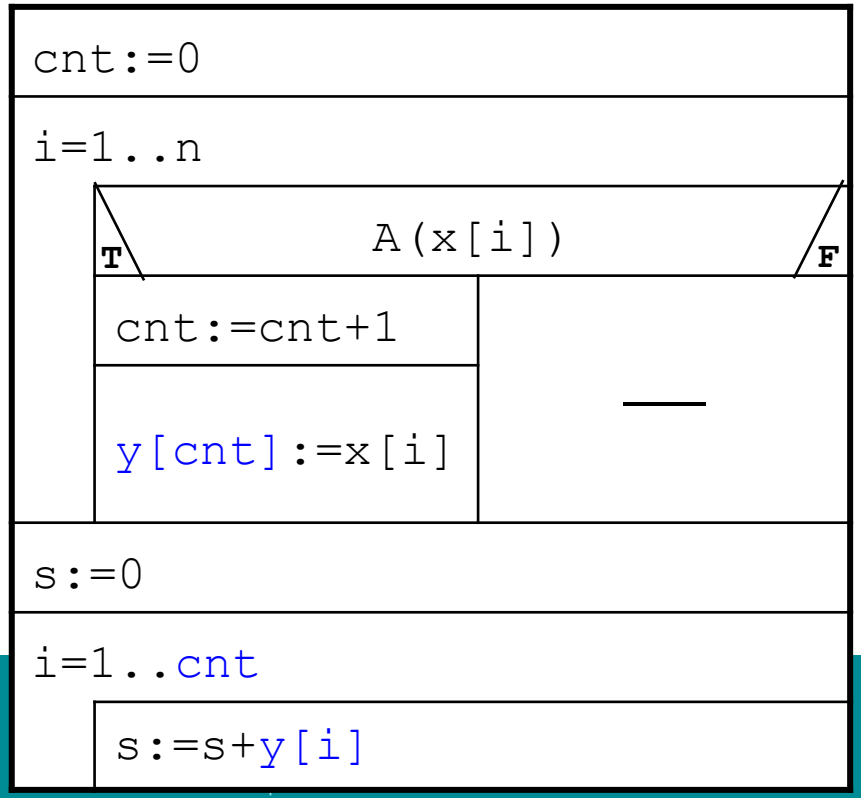
MIS and summation after each other:

```
In: n∈N, x∈H[1..n]
Aux: cnt∈N, y∈H[1..n]
Out: s∈H2
Pre: -
Post: (cnt,y)=MIS(i=1..n,A(x[i]),x[i]) and
       s=SUM(i=1..cnt,y[i])
```

MIS + summation

Problem: Sum of elements with a certain attribute – conditional summation.

```
Post: (cnt,y)=MIS(i=1..n,A(x[i]),x[i]) and
       s=SUM(i=1..cnt,y[i])
```



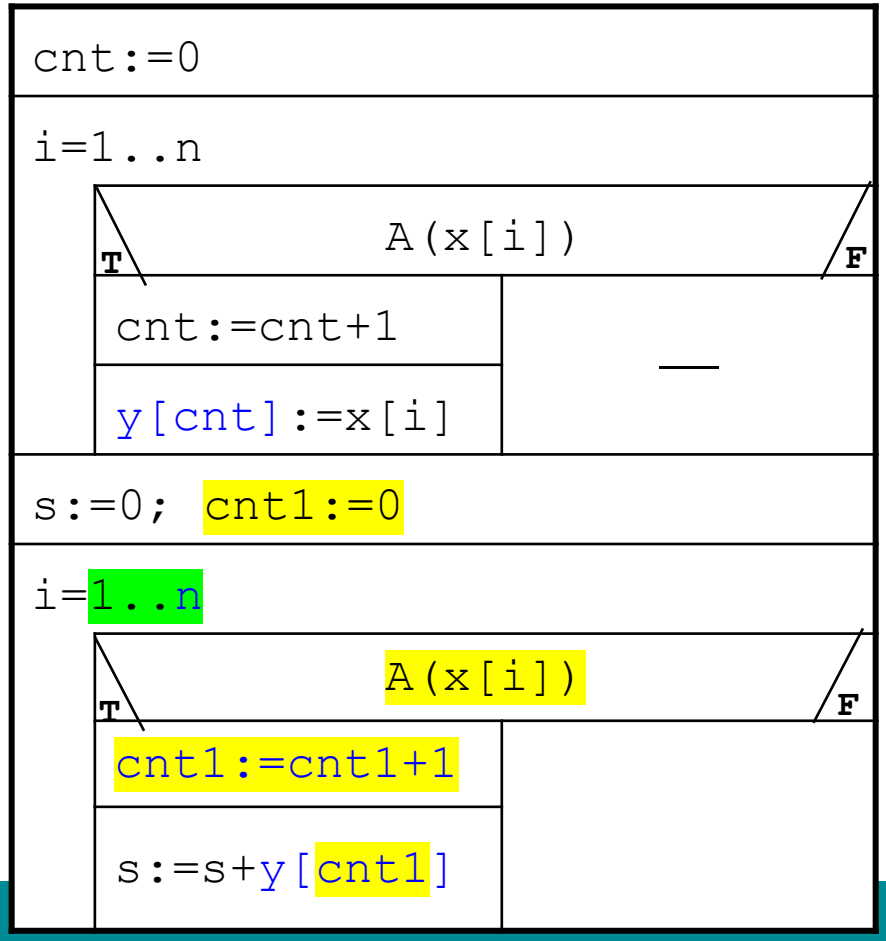
NB: the result is correct, but the algorithm is not really effective

MIS + summation

Let's apply program transformations

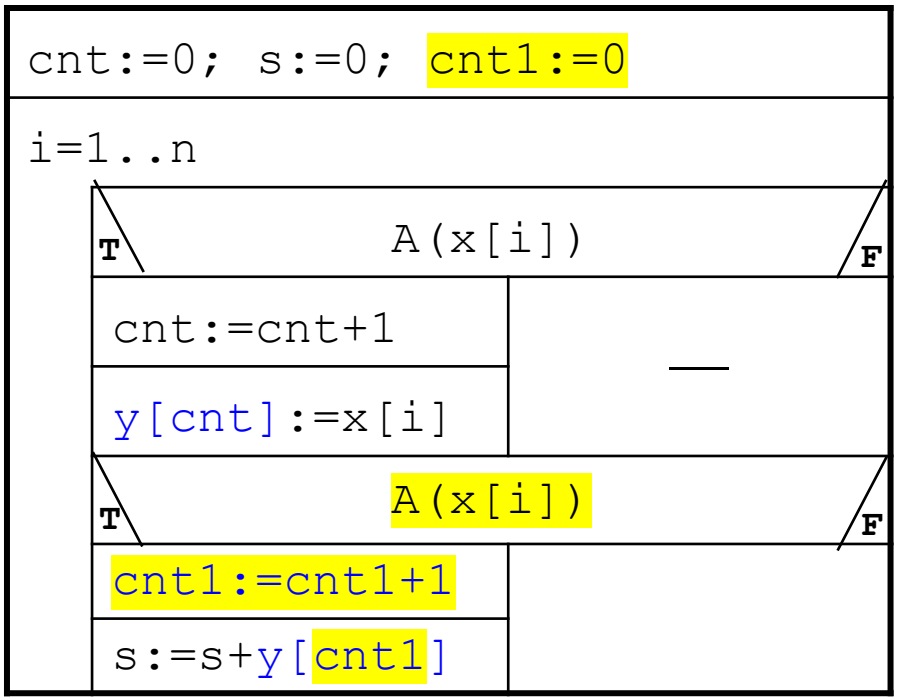
prepare/modify the loops to use the same number of loopsteps

(the new version is semanticaly equivalent with the old one



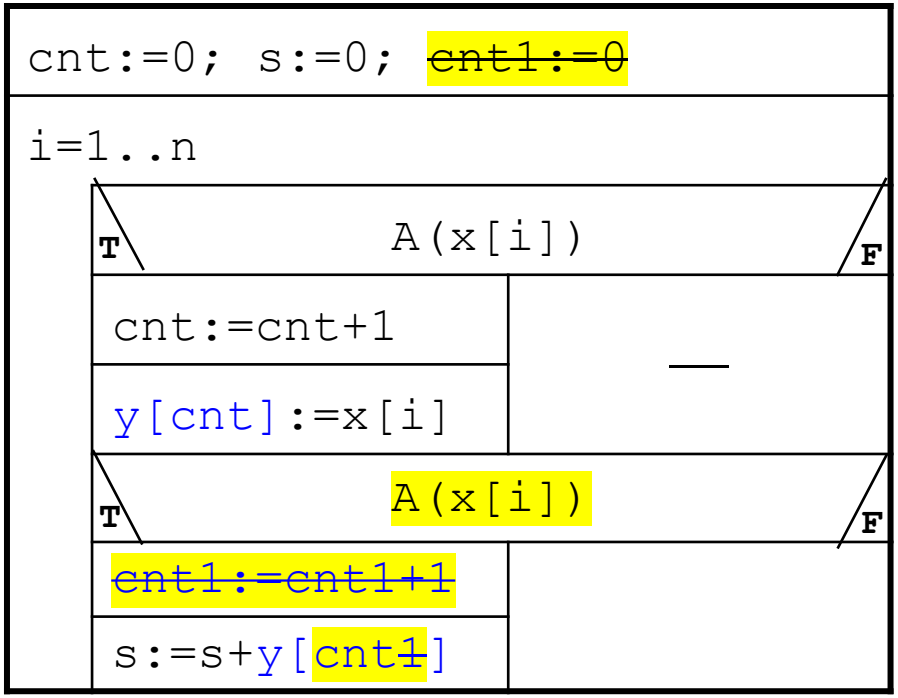
MIS + summation

Loop fusion with week connection…



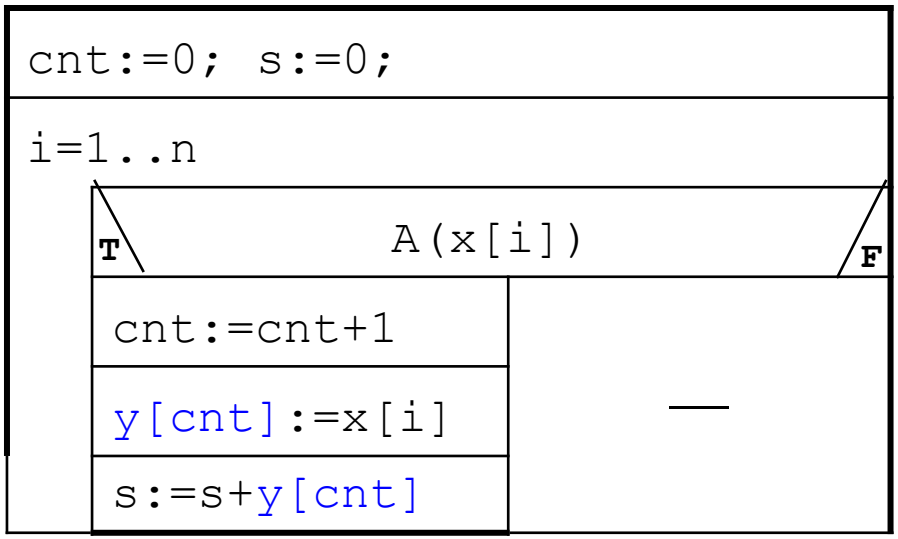
MIS + summation

cnt1 and cnt will be changed synchronously: remove db1



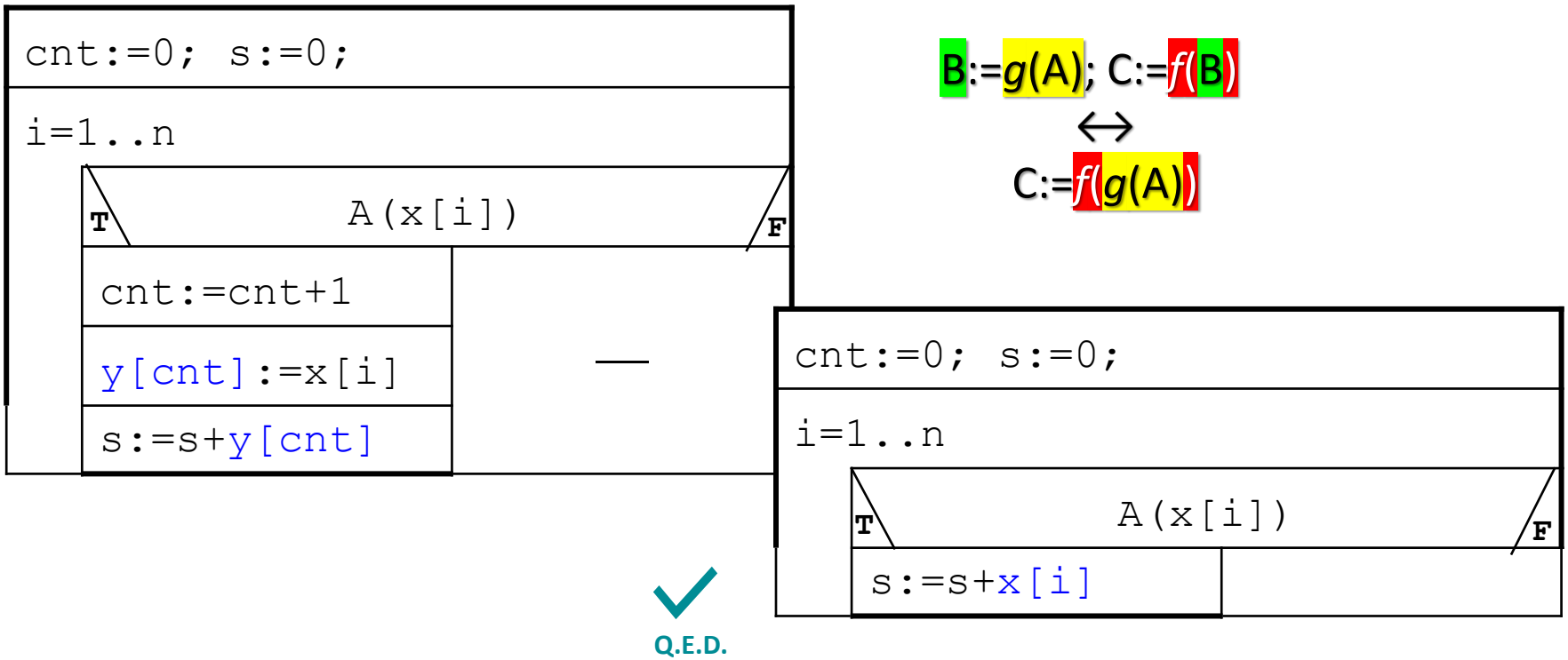
MIS + summation

fusion of the conditionals (the x[i] doesn't change the value)



MIS + summation

function composition



Post: s=SUM(i=1..n,x[i],A(x[i]))

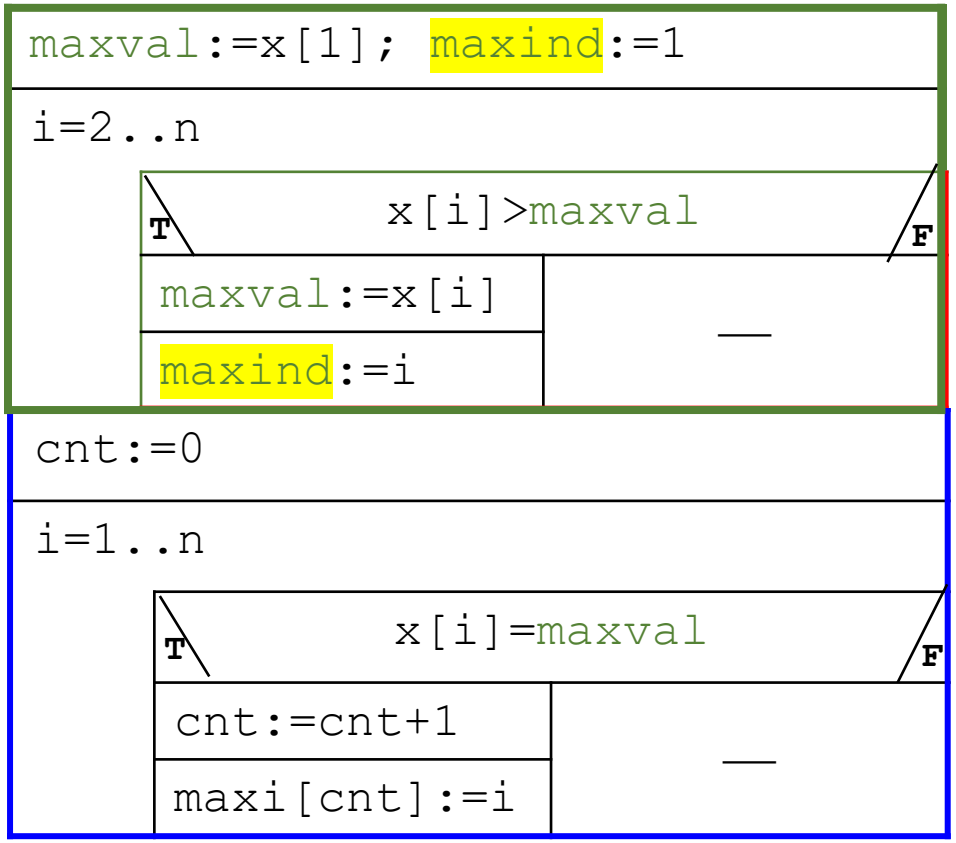
maximum selection + MIS

Problem: selecting **all** maximum elements.

```
In: n∈N, x∈Z[1..n]
Aux: maxval∈Z
Out: cnt∈N, maxi∈N[1..n]
Pre: n>0
Post: (,maxval)=MAX(i=1..n,x[i]) and
      (cnt,maxi)=MIS(i=1..n,x[i]=maxval,i)
```

maximum selection + MIS

```
Post: (,maxval)=MAX(i=1..n,x[i])
 and (cnt,maxi)=
     MIS(i=1..n, x[i]=maxval),i)
```

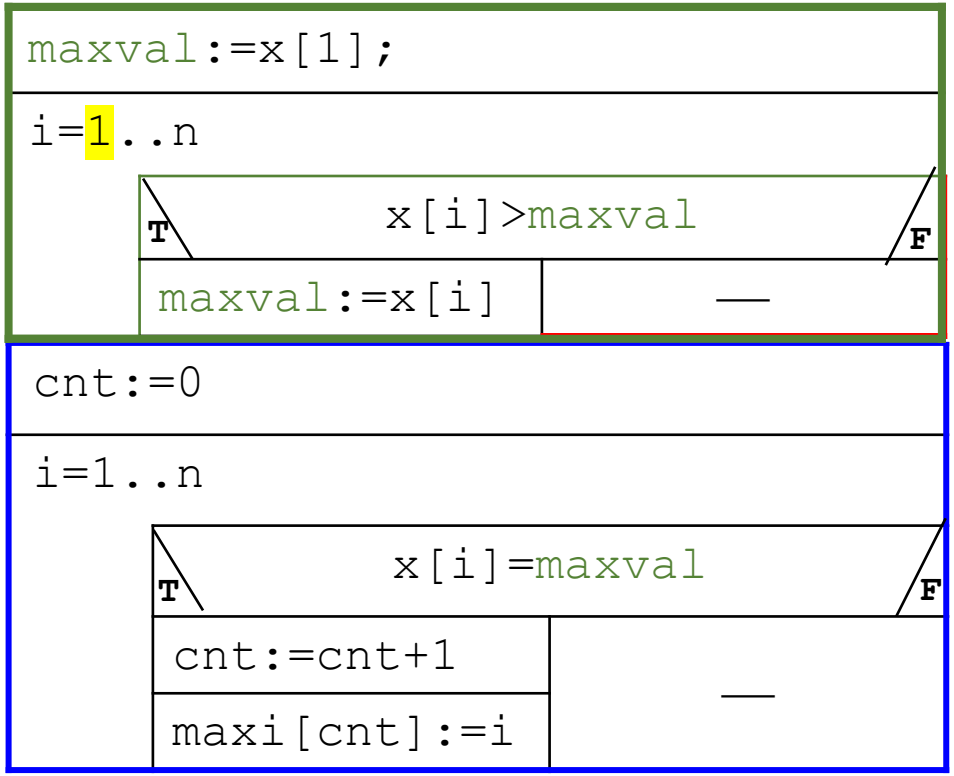


NB: the result is correct, but the algorithm is not really effective

maximum selection + MIS

Let's apply program transformations

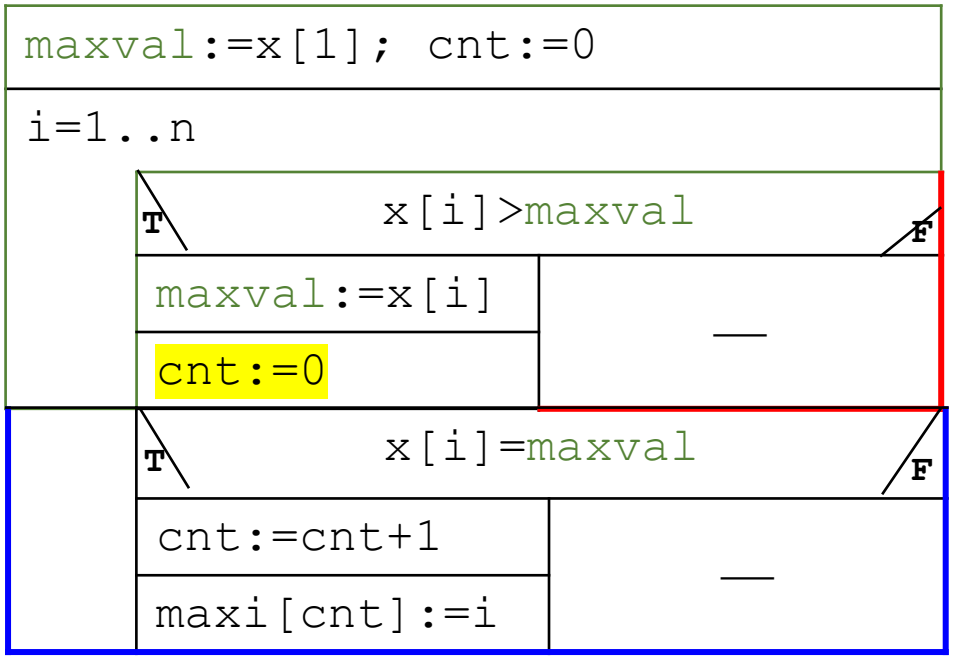
- maxind – unnecessary -> delete it
- syncronize the number of the loop steps (1..n) – only one extra operation (checking x[1]>x[1])



maximum selection + MIS

transformations: fusion of the loops and fusion of the conditionals

- if maxval will be changed – cnt need to be 0 again
- => also the 2nd conditional statement will be true – the 1st new max will be counted and stored



maximum selection + MIS

transformations: fusion of the *exclusive, full* conditionals

<table>
<tr><td colspan="3">maxval:=x[1]; cnt:=0</td></tr>
<tr><td>i=1..n</td><td colspan="2"></td></tr>
<tr><td></td><td>T x[i]>maxval</td><td>T x[i]=maxval</td></tr>
<tr><td></td><td>maxval:=x[i]</td><td>cnt:=cnt+1</td></tr>
<tr><td></td><td>cnt:=1</td><td rowspan="2">maxi[cnt]:=i</td></tr>
<tr><td></td><td>maxi[cnt]:=i</td></tr>
</table>

maximum selection + MIS

transformations: fusion of the *exclusive, full* conditionals

<table>
<tr><td colspan="3">maxval:=x[1]; cnt:=0; maxi[cnt]:=1</td></tr>
<tr><td colspan="3">i=2..n</td></tr>
<tr><td></td><td>T x[i]>maxval</td><td>T x[i]=maxval</td></tr>
<tr><td></td><td>maxval:=x[i]</td><td>cnt:=cnt+1</td></tr>
<tr><td></td><td>cnt:=1</td><td rowspan="2">maxi[cnt]:=i</td></tr>
<tr><td></td><td>maxi[cnt]:=i</td></tr>
</table>

the loop can start again from the 2nd element

decision+ counting

Problem: Are there **at least K** elements with the given attribute in a sequence?

```
In: n∈N, x∈Z[1..n]
Aux: cnt∈N
Out: exists∈L
Pre: n>0
Post: cnt=COUNT(i=1..n,A(x[i])) and exists=(cnt≥k)
```

counting

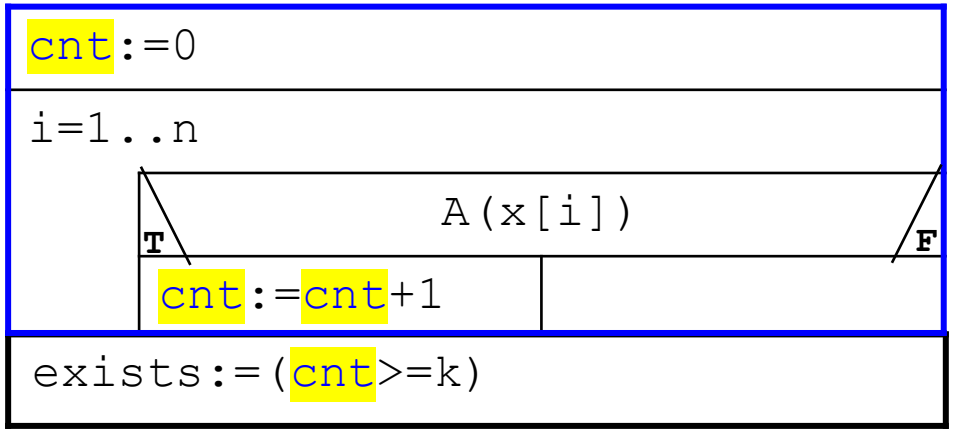
~~decision~~

decision+ counting

Problem: Are there **at least K** elements with the given attribute in a sequence?

…

Post: cnt=**COUNT**(i=1..n,A(x[i])) and exists=(cnt≥k)



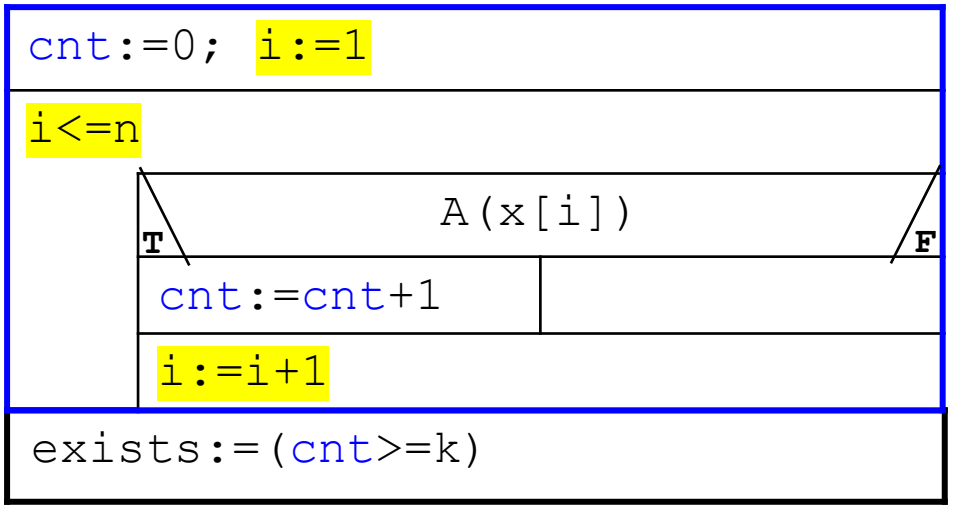
NB: the result is correct, but the algorithm is not really effective

-> if we counted K many element with A attribut, we dont need to count further

decision+ counting

Let's apply program transformations

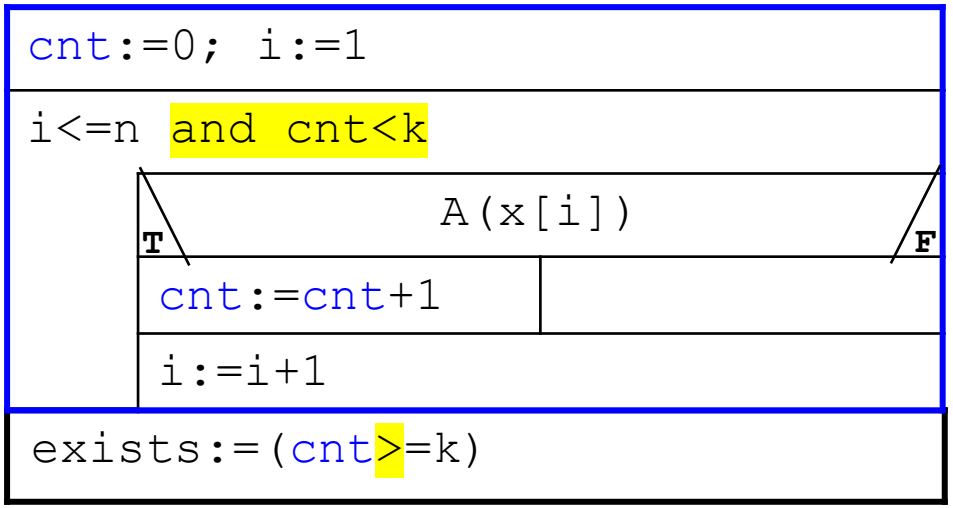
Modify the „counter" loop to conditional loop – create the oppurtunity finisching the loop earlier



decision+ counting

Let's apply program transformations

extend the loop condition with the checking of the k



The „passing" postcondition:

`Post: exists=DECISION(i=1..n,COUNT(j=1..i,A(x[j])=k)`

the algorithm also can be created only with programtransformations

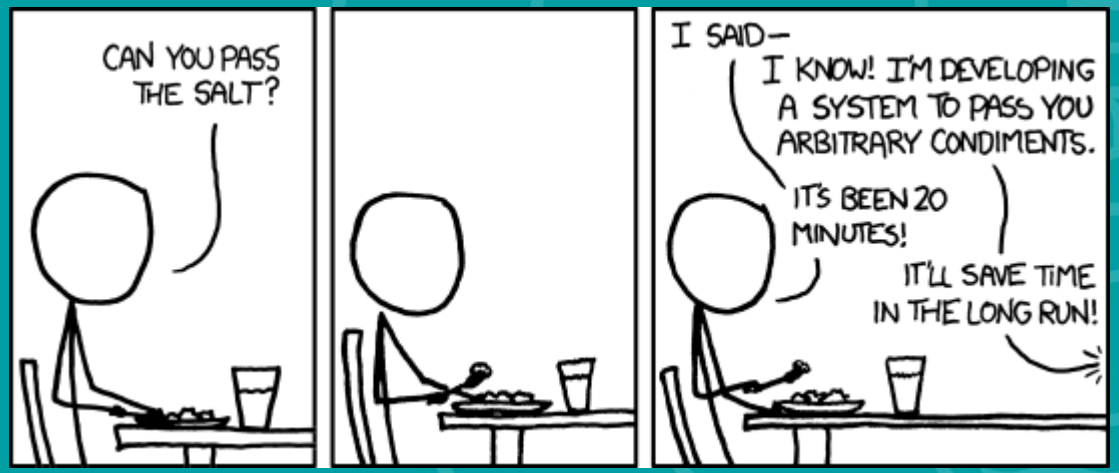
more combinations

- In a sequence, **which** is the **K.** element with a given attribute (if there are at least K elements with that attribute)?
- List all elements of a sequence that are before an element having attribute A. (If no element with the attribute, then all the elements.)

more combinations

- In a sequence, **which** is the **K.** element with a given attribute (if there are at least K elements with that attribute)? -> search + counting
- List all elements of a sequence that are before an element having attribute A. (If no element with the attribute, then all the elements.) -> search + copy

ELTE
FACULTY OF
INFORMATICS



Thank you for your attention!